Manual de ejecución de la Tarea 2

Procesamiento de artículos Crossref con Apache Spark

Adrian Rojas Rivera | Modalidad individual | 11 de octubre de 2026

Este manual explica cómo preparar los datos, ejecutar el programa y sus pruebas, y revisar las ocho salidas solicitadas. La cadena convierte JSON multilínea a Parquet y vuelve a leer ese Parquet antes de calcular las métricas. La ejecución comprobada procesó 31 458 documentos y las 23 pruebas incluidas pasaron.

1 Inicio rápido en el equipo preparado

Con Docker Desktop abierto, usar una nueva Terminal del Mac. El contenedor existente se llama bigdata-t2 y comparte la carpeta del proyecto mediante /t2.

```
docker exec -it bigdata-t2 /bin/bash
```

Dentro del contenedor, ejecutar una línea a la vez. El último comando reemplaza las salidas anteriores de esta tarea.

```
source /opt/venv/bin/activate
cd /t2
pytest
spark-submit --master 'local[2]' programa.py data/crossref --overwrite
```

Al finalizar debe aparecer “5/5 Finalizado” y la ubicación de output/resumen_ejecucion.json. Si el contenedor está detenido, ejecutar docker start bigdata-t2 desde el Mac antes de entrar. Jupyter puede permanecer abierto; el programa principal no depende del cuaderno.

Entorno comprobado

Componente	Versión o configuración
Sistema	Docker Desktop sobre macOS ARM64
Imagen y Java	bigdatafull y OpenJDK Temurin 17.0.20.1
Python y Spark	Python 3.10.12 y PySpark 4.2.0
Pruebas	pytest 9.1.1
Ejecución	local[2] y zona horaria UTC

La imagen ya preparada fue la utilizada en la validación. Las instrucciones para una instalación nueva aparecen en la página siguiente.

2 Preparar una copia nueva del proyecto

Obtener el código y los datos

Descargar el repositorio https://github.com/riverxh/Projects con Code y Download ZIP, descomprimirlo y abrir una Terminal en su carpeta T2. El dataset completo no se publica en el repositorio; se obtiene de L2/crossref.zip, material del curso. Los tests crean sus propios datos pequeños y pueden ejecutarse sin ese ZIP.

En el Mac utilizado, los datos ya están extraídos. Para otra copia, el siguiente comando extrae los JSON sin aplanar la carpeta crossref. Ajustar las rutas si el ZIP o el proyecto se guardan en otro lugar.

```
cd "/Users/adrian/Downloads/Big Data/T2"
mkdir -p data
unzip -q "/Users/adrian/Downloads/L2/crossref.zip" \
  'crossref/*.json' -d data
```

No repetir la extracción sobre los mismos archivos sin necesidad. Para comprobar la cantidad de JSON en esta copia:

```
python3 - <<'PY'
from pathlib import Path
print(len(list(Path("data/crossref").glob("*.json"))))
PY
```

El resultado observado fue 31 458. Cada archivo contiene un objeto con message; no es una fila de una tabla CSV.

Construir y abrir el contenedor

El Dockerfile incluido procede del Full Container ARM64 del curso. Se fijaron PySpark y pytest a las versiones comprobadas. La construcción necesita Internet y puede tardar varios minutos. Esta construcción nueva no se ejecutó durante la validación; las pruebas se hicieron en la imagen bigdatafull existente.

```
docker build -t tarea2-bigdata .
docker run --rm -it -v "$PWD:/t2" -w /t2 \
  tarea2-bigdata /bin/bash
```

El montaje conserva los resultados en la carpeta del Mac. Dentro del contenedor:

```
source /opt/venv/bin/activate
cd /t2
pytest
spark-submit --master 'local[2]' programa.py data/crossref
```

Si ya existe una de las ocho carpetas de salida, usar --overwrite solo cuando se quiera reemplazarla. En un equipo sin Docker, se requiere un entorno compatible con Python 3.10, Java 17 y las dependencias fijadas en requirements.txt; esa alternativa no fue validada aquí.

3 Organización y transformación de los datos

Archivo	Responsabilidad
programa.py	Argumentos, sesión Spark, escritura Parquet, relectura y coordinación de métricas.
functions/esquemas.py	Tipos explícitos de entrada y salida, incluidas las estructuras anidadas.
functions/procesamiento.py	Lectura, proyección, agrupaciones, referencias, percentiles y escritura.
tests/ y pytest.ini	Pruebas funcionales, excepciones e integración desde la raíz de T2.
Dockerfile y requirements.txt	Preparación del entorno y versiones de PySpark y pytest.
doc/	Manual Word, guion y evidencias de validación.

El flujo es JSON → proyección de nueve columnas → escritura Parquet → nueva lectura Parquet → agregaciones → CSV y JSON. El programa comprueba que la cantidad de filas leídas del Parquet sea igual a la cantidad inicial. No aplana los autores ni las referencias en el archivo Parquet.

Correspondencia de columnas

Origen dentro de message	Columna Parquet	Tipo
DOI	doi	string
author	author	array de struct
institution	inst	array de struct
group-title	groupTitle	string
created.date-time	createdDate	string
prefix	prefix	string
reference	reference	array de struct
link	link	array de struct
subtype	subtype	

string

author conserva ORCID, affiliation, authenticated-orcid, family, given, name, sequence y suffix. affiliation e inst contienen objetos con name; authenticated-orcid es booleano y los demás campos simples son texto.
reference conserva los 14 campos del esquema: DOI, ISSN, article-title, author, doi-asserted-by, first-page, issn-type, issue, journal-title, key, unstructured, volume, volume-title y year. link conserva URL, content-type, content-version e intended-application. Los campos opcionales ausentes permanecen nulos con su tipo definido.

4 Ejecutar y revisar las ocho salidas

Desde /t2 se admite una carpeta, varios archivos o un patrón. El directorio evita los límites de longitud de argumentos al trabajar con miles de archivos.

```
spark-submit --master 'local[2]' programa.py data/crossref
spark-submit --master 'local[2]' programa.py data/crossref/*.json
```

Son dos formas alternativas de ejecutar la misma tarea; no es necesario ejecutar ambas. Las rutas relativas parten del directorio actual. Se puede separar una demostración pequeña con --output output_muestra.

Carpeta dentro de output
Formato y columnas
crossref-parquet
Parquet con las nueve columnas de la sección anterior.
articles_month_year
CSV: created_year, created_month, total_articles.
research_group
CSV: group_title, total.
research_areas_per_person
JSON: full_name, research_areas, total.
person_most_references
JSON: full_name, total_references; máximo de una persona.
percentil_25
CSV: total_references; una fila con P25.
percentil_50
CSV: total_references; una fila con P50.
percentil_75
CSV: total_references; una fila con P75.

Cada carpeta CSV o JSON contiene un único archivo de datos part-*; _SUCCESS y los archivos ocultos son metadatos. Parquet puede contener varios archivos. El JSON de Spark contiene un objeto por línea, sin envolver el conjunto en un arreglo. Los CSV llevan encabezado y representan los nulos como null.

Comprobación de resultados

```
cat output/resumen_ejecucion.json
cat output/person_most_references/part-*.json
cat output/percentil_25/part-*.csv
cat output/percentil_50/part-*.csv
cat output/percentil_75/part-*.csv
head -n 8 output/articles_month_year/part-*.csv
```

En Finder, abrir /Users/adrian/Downloads/Big Data/T2/output. Esta ubicación corresponde a /t2/output dentro del contenedor. No buscar /t2 directamente en la Terminal del Mac: esa ruta existe dentro de Docker.

5 Reglas de cálculo e interpretación

Artículos por mes y por grupo

El mes y el año se extraen de createdDate en UTC. Se cuenta una fila por documento, sin multiplicar por autores. Una fecha ausente o inválida se conserva en el grupo con año y mes nulos. La salida mensual se ordena por año y mes ascendentes.

Por grupo se cuentan DOI distintos no nulos y no vacíos. Un DOI repetido dentro del mismo grupo contribuye una sola vez. El grupo nulo se conserva; no se deduplican globalmente los documentos para otras métricas. La salida se ordena por total descendente y nombre de grupo ascendente.

Nombres y áreas de investigación

El nombre se forma concatenando given y family con un espacio y aplicando trim al resultado. Si queda vacío se usa unknown. Un autor nulo se omite; un arreglo de autores vacío o ausente no crea una persona. Un nombre repetido en el mismo artículo contribuye una vez.

Cada persona recibe el conjunto de groupTitle válidos de sus artículos. Se excluyen grupos nulos o en blanco; la lista se ordena alfabéticamente y total es su tamaño. La salida se ordena por total descendente y nombre ascendente. Una persona sin grupos válidos puede tener una lista vacía y total cero. Los homónimos se agrupan por el nombre; no se aplica identificación por ORCID.

Referencias por persona y máximo

Se usan los objetos de reference cuyo DOI existe y no queda vacío al aplicar trim. El modo predeterminado cuenta DOI distintos por persona entre todos sus artículos. No cuenta citas recibidas ni sustituye el cálculo por reference-count. Cada referencia puede contribuir a cada coautor; no se divide entre ellos.

La decisión de usar referencias distintas se apoya en el ejemplo del enunciado. El conteo independiente de Florian Krammer encontró 100 artículos, 3869 entradas con DOI y 2701 DOI distintos. Se conservan las mayúsculas originales del DOI. El texto no exige explícitamente deduplicar estas referencias, por lo que esta interpretación debe confirmarse con el profesor.

Por defecto se excluyen de referencias las personas sin DOI válidos y el nombre sintético unknown, que puede reunir autores diferentes. unknown sí permanece en las áreas de investigación. En empate del máximo se elige el nombre ascendente. Las opciones permiten contar todas las entradas o incluir los casos excluidos; no son aclaraciones confirmadas del docente.

6 Percentiles y opciones de ejecución

Los percentiles se calculan sobre todos los totales de referencias por persona, antes de seleccionar el máximo. La población predeterminada comprende 194 981 nombres identificados con al menos una referencia válida en el dataset comprobado. Los puntos 6 a 8 mencionan el punto 4, pero hablan de referencias; esta implementación toma como base los totales de referencias del punto 5.

El método continuous usa el percentil exacto interpolado de Spark: para los valores ordenados se toma la posición (n−1) × p y se interpola entre vecinos cuando corresponde. La alternativa nearest-rank selecciona el valor observado en el rango ceil(n × p), contando desde uno. Por ejemplo, para 1, 2, 3 y 4, P25 continuo es 1.75 y nearest-rank es 1. El método y la población no quedan definidos completamente en el enunciado.

Opción

Efecto

--output RUTA

Cambia output por otra carpeta.

--overwrite

Reemplaza las ocho salidas y actualiza el resumen.

--reference-count-mode entries

Cuenta todas las entradas válidas; el valor por defecto es distinct-doi.

--include-zero-references

Incluye nombres con cero referencias válidas.

--include-unknown-references

Incluye el grupo sintético unknown en referencias.

--percentile-method nearest-rank

Cambia continuous por el método de rango observado.

--no-csv-header

Escribe los CSV sin encabezado.

--parquet-partitions N

Solicita hasta N archivos Parquet; por defecto 8. Debe ser positivo.

Ejemplo de configuración alternativa

```
spark-submit --master 'local[2]' programa.py data/crossref \
  --output output_alternativo --reference-count-mode entries \
  --include-zero-references --percentile-method nearest-rank
```

Esta alternativa puede cambiar el máximo y los percentiles. No compararla con los valores predeterminados sin revisar la configuración guardada en resumen_ejecucion.json. Para consultar todas las opciones:

```
spark-submit programa.py --help
```

Si la población está vacía, el máximo tiene cero registros y los percentiles son nulos. JSON malformado, ausencia de message o entrada sin artículos producen error. La escritura no es transaccional: un fallo puede dejar salidas parciales; corregir la causa y repetir con --overwrite.

7 Pruebas y validación observada

Las pruebas se ejecutan desde la raíz de T2 con pytest. La fixture de sesión prepara Spark local[2] y lo cierra al terminar. Los casos usan datos pequeños con valores esperados definidos por separado, sin requerir el dataset completo.

```
pytest
```

El 11 de octubre de 2026 se observaron 23 passed in 14.26s. La ejecución completa terminó con código de salida 0. Estas comprobaciones fueron realizadas por el asistente en el entorno indicado; no implican que el estudiante ya haya repetido o revisado personalmente todas las pruebas.

Comprobación	Resultado observado
JSON leídos y filas Parquet releídas	31 458 en ambas etapas
Archivos de datos Parquet	8
Archivos de datos por CSV y JSON	1 por salida
Mayor total de referencias distintas	Florian Krammer con 2701
Percentiles continuos P25 P50 P75	21.0, 36.0 y 64.0
Contraste independiente con el ZIP original	Meses, grupos, áreas y orden, máximo y percentiles coinciden

Los tests cubren lectura multilínea y múltiples rutas, esquema anidado, campos ausentes, rutas y JSON inválidos, fechas, duplicados, coautores, nombres vacíos, grupos, referencias, ceros, empates, población vacía, ambos métodos de percentiles, archivo único, relectura y reemplazo. La evidencia queda en doc/Validacion.md, doc/Validacion_dataset.json y doc/Comprobacion_referencias.json.

Resolver problemas frecuentes

PATH_NOT_FOUND: comprobar la ruta y el directorio actual con pwd. Dentro de /t2 usar data/crossref o /t2/data/crossref; ./t2/data desde /t2 buscaría /t2/t2/data. command not found: activar /opt/venv/bin/activate. Contenedor detenido: iniciarlo desde el Mac. Salidas existentes: elegir una carpeta nueva o usar --overwrite intencionalmente.

Memoria: el programa reduce a 128 filas los lotes Parquet, conserva temporalmente la proyección JSON en disco y evita cachear todas las columnas. La ejecución comprobada usó el heap Java predeterminado de 1 GiB. Los avisos de biblioteca nativa de Hadoop no equivalen por sí solos a un fallo; revisar si terminó el programa y se generó el resumen actual.

Las 23 pruebas locales y el contraste del dataset no garantizan las pruebas adicionales del profesor, especialmente ante las interpretaciones pendientes de confirmar.

Fuentes y procedencia del trabajo

Materiales utilizados

1. Enunciado 2026-02-M4-T2.md de L3. Fuente de las ocho salidas, esquema, pruebas, manual, video y reglas de entrega.

2. Full Container de L1, Dockerfile.arm64. Base de la configuración Docker incluida; se fijaron PySpark 4.2.0 y pytest 9.1.1. Las demás dependencias siguen la configuración del material de clase.

5. Apache Spark 4.2.0. JSON Files y pyspark.sql.functions.percentile. La lectura multilínea y el percentil exacto se documentan en estas referencias.
https://spark.apache.org/docs/4.2.0/sql-data-sources-json.html
https://spark.apache.org/docs/4.2.0/api/python/reference/pyspark.sql/api/pyspark.sql.functions.percentile.html

6. Política para el Uso de Inteligencia Artificial Generativa, versión 1.0, septiembre de 2026.

Repositorio y entrega

Repositorio de trabajo: https://github.com/riverxh/Projects, carpeta T2. Incluye código, pruebas, configuración, manual Word, guion y evidencias; excluye el dataset completo y las salidas voluminosas. Los resultados se regeneran siguiendo este manual.